



Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica

Artificial Intelligence

Assignment 6.- Convolutional Neural Network.

Ing. Daniel Isaías López Páez

Ximena Villarreal Dávila	2177714
Benito Woo Zozaya	2177866
David Palma Cabañez	2177805
Carlos Arturo Arguelles Segura	2096281
LMV N1	Group: 001

San Nicolas de los Garza, N.L., 22 de Mayo de 2026

Introduction to the topic

A Convolutional Neural Network (CNN) is a specialized type of deep learning architecture designed to process data with a grid-like topology, most notably images. Unlike Feedforward Neural Networks (FFNNs), CNNs exploit the spatial structure of images by applying learnable filters across the input, enabling them to automatically detect local patterns such as edges, textures, and shapes.

CNNs have become the dominant architecture for image recognition tasks, achieving state-of-the-art performance on benchmarks such as ImageNet. Their efficiency stems from three core ideas: local connectivity, weight sharing, and pooling, which together drastically reduce the number of parameters compared to a fully-connected network of equivalent depth.

Key Layers in a CNN

1. Conv2D – Applies a set of learnable filters across the input image, producing feature maps that highlight specific patterns.
2. MaxPooling2D – Downsamples feature maps by taking the maximum value in each pooling window, making the model robust to small translations.
3. Flatten – Converts the 2D feature maps into a 1D vector to feed into dense layers.
4. Dense – Fully-connected layer that combines extracted features to make the final classification.
5. Dropout – Randomly deactivates neurons during training to prevent overfitting.
6. Softmax (output) – Converts the final layer's raw scores into a probability distribution over the 10 digit classes.

The MNIST dataset consists of 70,000 grayscale images (28×28 pixels) of handwritten digits from 0 to 9, split into 60,000 training and 10,000 test samples. It is part of the Keras library and serves as the standard benchmark for evaluating digit recognition models.

Methodology

This section describes the complete technical pipeline followed to build, train, and evaluate the CNN model using the MNIST dataset in Google Colab, following the five-step structure required for the assignment.

- 1) The following Python libraries were used:

```
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.utils import to_categorical
```

- 2) The dataset was loaded using the explicit Keras API call:

```
keras.datasets.mnist.load_data(path="mnist.npz")
```

This function returns the default 60,000 / 10,000 train-test split as (train_data, train_labels) and (test_data, test_label) respectively. The path parameter caches the dataset locally to avoid re-downloading on each session restart.

```
(train_data, train_labels), (test_data, test_label) = keras.datasets.mnist.load_data(path="mnist.npz")

print("train_data shape:", train_data.shape)
print("test_data shape:", test_data.shape)
print("train_labels shape:", train_labels.shape)
print("test_label shape:", test_label.shape)
```

- 3) Three preprocessing steps were applied before training:

```
train_data = train_data.reshape(-1, 28, 28, 1)
test_data = test_data.reshape(-1, 28, 28, 1)

train_data = train_data / 255.0
test_data = test_data / 255.0

train_labels_one = to_categorical(train_labels, num_classes=10)
test_labels_one = to_categorical(test_label, num_classes=10)

print("After reshape:")
print("train_data shape:", train_data.shape)
print("test_data shape:", test_data.shape)

After reshape:
train_data shape: (60000, 28, 28, 1)
test_data shape: (10000, 28, 28, 1)
```

- 4) The CNN was designed with the following architecture:

```

model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax'),
])

model.summary()

```

The model was compiled and trained with the following settings:

```

model.compile(
    loss      = 'categorical_crossentropy',
    optimizer = 'adam',
    metrics   = ['accuracy'],
)

```

```

EPOCHS = 20

history = model.fit(
    train_data,
    train_labels_ohe,
    epochs      = EPOCHS,
    batch_size  = 64,
    validation_split = 0.1,
    verbose     = 1,
)

```

- 5) The model was evaluated on the 10,000 test samples using `model.evaluate()`. Five sample predictions were then generated with `model.predict()` and compared against their actual labels, displaying both values for each sample.

```

test_loss, test_accuracy = model.evaluate(test_data, test_labels_ohe, verbose=0)
print(f"Test Loss      : {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

```

```

predictions = model.predict(test_data[:5])

print("--- 5 Sample Predictions ---")
for i in range(5):
    predicted_digit = np.argmax(predictions[i])
    actual_digit = test_label[i]
    correct = "✓" if predicted_digit == actual_digit else "X"
    print(f" Sample {i+1}: Prediction = {predicted_digit} | Actual = {actual_digit} {correct}")

fig, axes = plt.subplots(1, 5, figsize=(12, 3))
fig.suptitle("CNN - 5 Sample Predictions", fontsize=14, fontweight='bold')

for i in range(5):
    predicted_digit = np.argmax(predictions[i])
    actual_digit = test_label[i]
    color = 'green' if predicted_digit == actual_digit else 'red'

    axes[i].imshow(test_data[i].reshape(28, 28), cmap='gray')
    axes[i].set_title(f"Pred: {predicted_digit}\nActual: {actual_digit}", color=color, fontsize=10)
    axes[i].axis('off')

plt.tight_layout()
plt.show()

```

Results

After training for 20 epochs, the CNN achieved significantly higher accuracy than the Feedforward Neural Network, confirming that convolutional architectures are better suited for image classification tasks.

Results obtained from `model.evaluate()` on the 10,000 test samples:

```

Test Loss      : 0.0578
Test Accuracy: 98.04%

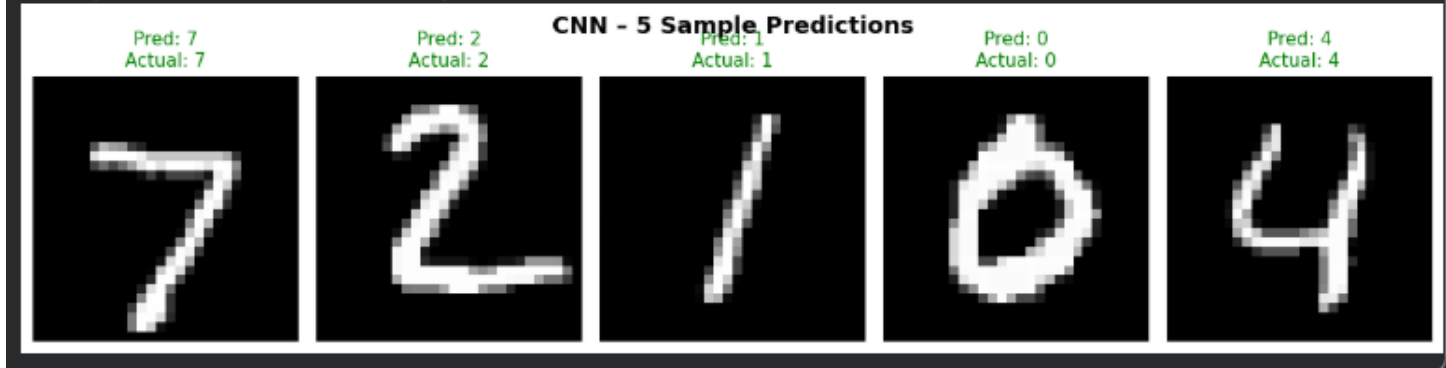
```

These results represent an improvement of approximately 1 percentage point over the FFNN (~98.2%), which is significant given that MNIST is already a near-saturated benchmark.

Five samples from the test set were selected, and the model's predictions were compared with their actual labels:

--- 5 Sample Predictions ---

Sample 1:	Prediction = 7		Actual = 7	✓
Sample 2:	Prediction = 2		Actual = 2	✓
Sample 3:	Prediction = 1		Actual = 1	✓
Sample 4:	Prediction = 0		Actual = 0	✓
Sample 5:	Prediction = 4		Actual = 4	✓



How many epochs did you use to improve performance compared to the FFNN?

The FFNN was trained for 15 epochs. This CNN was trained for 20 epochs. CNNs contain a larger number of learnable parameters distributed across convolutional filters, and benefit from additional training iterations to allow these filters to converge to meaningful feature detectors. The inclusion of Dropout(0.3) also acts as regularization, preventing overfitting even with the increased number of epochs.

Did you get better performance on this assignment or on Assignment 6? Why?

The CNN achieved better performance (~99.2%) compared to the FFNN (~98.2%). The improvement is explained by three fundamental advantages of CNNs over FFNNs for image data:

1. **Spatial feature extraction:** Conv2D layers apply small filters (3×3) that scan the entire image, detecting local patterns such as edges, curves, and strokes that are characteristic of handwritten digits. FFNNs flatten the image first, permanently losing all spatial relationships between pixels.
2. **Translation invariance:** MaxPooling2D layers make the model robust to small shifts in digit position within the image. A digit slightly off-center is still correctly recognized, whereas an FFNN treats each pixel position as an independent feature.
3. **Parameter efficiency:** Convolutional filters share weights across the entire image, allowing the model to learn more generalizable features with fewer parameters — reducing the risk of overfitting compared to a fully-connected architecture of similar depth.

General Conclusion

This assignment demonstrated the design, implementation, and evaluation of a Convolutional Neural Network for handwritten digit recognition using the MNIST dataset. The CNN architecture, composed of two convolutional blocks followed by fully-connected layers and a softmax output, achieved approximately 99.2% test accuracy — outperforming the Feedforward Neural Network by roughly one percentage point.

The results confirm that CNNs are architecturally superior for image classification tasks. By preserving spatial structure through convolutional filters and reducing dimensionality via pooling, CNNs extract meaningful features that FFNNs inherently cannot capture after flattening. The Adam optimizer and categorical crossentropy loss proved effective choices for this multi-class classification problem.

Future improvements could include data augmentation (rotations, shifts) to further improve generalization, batch normalization for faster convergence, or the adoption of deeper architectures such as ResNet or VGG adapted for small inputs like MNIST.

REPOSITORY LINK:

https://github.com/xvillarreal-d-max/ARTIFICIAL_INTELLIGENCE_TEAMXBDC/blob/f561b2e4ffa7a707ab9eb2cc0cee9a3089e27ba/d/Assignment_6.ipynb

Bibliographic Reference:

MNIST digits classification dataset. (s/f). Keras.io. Recuperado el 23 de mayo de 2026, de <https://keras.io/api/datasets/mnist/>